Table of Contents

Abstract

This final report presents the design and implementation of the AI-Powered Personalized Meal Recommendation Platform, a full-stack web application developed as a CENG 408 graduation project. The project addresses a practical problem in digital meal planning: many existing systems either provide static recipe suggestions without considering the user's real inventory and dietary context, or rely directly on generative AI outputs without sufficient validation. The platform was designed to combine structured nutritional data, user preferences, household inventory, consumption history, and optional AI-assisted explanation in a controlled and verifiable recommendation workflow.

The system is implemented with a modular Java 21 and Spring Boot backend, a React and TypeScript frontend, and a PostgreSQL relational database. Authentication and authorization are handled through Keycloak-based OAuth2/OIDC integration, while object storage is supported through MinIO. The backend separates infrastructure, domain, application, and utility concerns, allowing external integrations such as AI providers and storage services to remain isolated from core business logic. The frontend follows a feature-oriented structure and provides user workflows for profile management, inventory management, recipe discovery, recommendation generation, consumption logging, notifications, and administrative operations.

The recommendation methodology follows a hybrid approach. First, the backend filters recipes against hard constraints such as diet type and allergies. Then, deterministic scoring ranks candidate recipes or menu alternatives using signals such as inventory match, taste preferences, cravings, nutrition proximity, ratings, preparation practicality, and recent usage history. When an AI provider is selected, the AI layer operates on an already validated candidate pool and returns structured explanations or menu selections. If the user selects the free mode or if the AI service fails, the system falls back to deterministic recommendations, preserving functionality without depending entirely on external AI services.

The project also includes a data preparation pipeline for cleaning, normalizing, validating, and importing recipe, ingredient, nutrition, and unit conversion data. Nutrition calculations are based on ingredient-level macro values and gram conversions, with support for practical household units. Quality assurance is supported through layered tests, controller and service verification, integration-oriented test infrastructure, and build-time checks. Overall, the delivered platform demonstrates an end-to-end software engineering solution for personalized nutrition planning, collaborative inventory usage, explainable AI-assisted recommendations, and maintainable full-stack application design.

1. Introduction

Digital nutrition applications have become increasingly common, but many of them still treat meal planning as an isolated recommendation problem. A user is usually expected to enter a goal, select a diet label, and receive a list of recipes that may or may not match the food they already have, their household context, their recent consumption, or their real nutritional needs. At the same time, the recent popularity of large language models has encouraged a second type of solution: applications that forward user input directly to an AI service and present the generated

response as a meal recommendation. Although this can produce fluent text, it also introduces risks such as hallucinated ingredients, unreliable nutrition values, weak allergy handling, and recommendations that are disconnected from the application's database.

The AI-Powered Personalized Meal Recommendation Platform was developed to address these limitations through a more controlled engineering approach. The project does not treat artificial intelligence as a replacement for domain logic. Instead, it combines deterministic backend filtering, relational data, nutritional calculations, inventory-aware scoring, and optional AI-generated explanation. The purpose is to recommend meals that are not only plausible in natural language, but also grounded in the user's profile, available ingredients, dietary restrictions, and consumption history.

The project evolved from an initial CENG 407 design and research phase into a working CENG 408 implementation. During the implementation phase, the scope expanded beyond basic recipe recommendation. The final system includes user profile management, Keycloak-based authentication, shared inventory groups, ingredient and unit conversion support, recipe lifecycle management, menu recommendation, consumption logging, notification workflows, administrative features, and a Python-based data preparation pipeline. This broader scope reflects the fact that meal recommendation is not only an algorithmic task; it is also a product workflow that depends on reliable data, usable interfaces, secure identity management, and maintainable backend architecture.

From a software engineering perspective, the project is structured as a full-stack web application. The backend is implemented with Java 21 and Spring Boot using a modular organization that separates infrastructure concerns, domain logic, application delivery, and utility code. The frontend is implemented with React and TypeScript using feature-based modules and typed service abstractions. PostgreSQL provides relational persistence, Flyway manages database evolution, Keycloak handles identity and access control, and MinIO supports object storage for media-related workflows. These technology choices were selected to support maintainability, testability, and realistic deployment practices within the constraints of a graduation project.

1.1 Human-in-the-Loop Agentic Co-Engineering

The development methodology of the project followed a Human-in-the-Loop Agentic Co-Engineering approach. In this model, AI agents were positioned as junior developer assistants that accelerated implementation, code generation, refactoring support, and documentation drafting. The project team, in contrast, acted as the technical project manager and senior software architect responsible for architectural decisions, domain rules, validation, and final code acceptance. The high-level abstractions of the system, module boundaries, database relationships, security model, recommendation constraints, and strict business rules were designed by the team before implementation tasks were delegated to AI-assisted workflows.

During development, these engineering decisions were transformed into strict prompts, implementation constraints, and review criteria. AI-generated code was not accepted blindly; it was inspected, corrected, tested, and adapted according to the project's layered architecture, dependency boundaries, and maintainability requirements. When generated code violated architectural rules, introduced dependency leakage, weakened domain separation, or produced behavior inconsistent with the intended workflow, the team intervened with engineering

judgment and revised the implementation manually. Therefore, AI was used as a productivity and exploration tool rather than as an autonomous decision maker. This methodology reflects the broader philosophy of the project: AI can assist software engineering, but human engineers must define the system boundaries, evaluate correctness, and remain accountable for the delivered product.

This report documents the implemented system rather than only the original idea. It explains the problem context, requirements, architecture, data model, security design, recommendation methodology, nutrition and unit conversion logic, inventory workflows, frontend structure, testing approach, deployment environment, risks, and future improvements. The report also highlights where the system deliberately avoids overreliance on AI by validating and ranking candidate recipes before any external model is used.

2. Problem Definition and Motivation

Meal planning is a daily decision-making problem shaped by several constraints at the same time. A useful recommendation must consider what the user can eat, what the user wants to eat, what ingredients are available, what nutritional target should be respected, and whether the recommendation can be trusted. Many applications address only one part of this problem. Recipe databases usually focus on search and filtering. Calorie trackers usually focus on logging after a meal has already been consumed. AI chat interfaces can generate meal ideas, but they often lack direct access to verified inventory, recipe, nutrition, and user-profile data. As a result, the user's actual cooking context remains fragmented across different tools.

The first major problem is the lack of inventory awareness. In real households, meal decisions often start with available ingredients rather than abstract dietary goals. A user may have vegetables, grains, leftovers, or low-stock items that should influence what they cook next. If a recommendation system ignores inventory, it may suggest recipes that are nutritionally valid but impractical. This increases friction, creates unnecessary shopping needs, and reduces the chance that the recommendation will be used.

The second problem is personalization quality. Diet type, allergies, disliked ingredients, daily calorie targets, physical profile data, and recent consumption history all affect whether a meal is appropriate. These constraints do not have equal importance. Allergies and incompatible diet types are hard constraints because violating them can make a recommendation unsafe or unusable. Preferences and cravings are softer signals because they should influence ranking without always eliminating otherwise useful recipes. A recommendation platform therefore needs a methodology that separates safety-related filtering from preference-based scoring.

The third problem is nutritional reliability. Meal recommendation systems must not treat calories and macronutrients as decorative information. If nutrition values are calculated from inconsistent ingredient quantities or unsupported household units, the output becomes unreliable. Practical cooking data often contains units such as glass, spoon, slice, clove, bowl, or piece. These units must be normalized into gram-based values before nutrition totals can be calculated consistently. This is especially important for a system that connects recipe recommendation with consumption tracking and daily nutrition analysis.

The fourth problem is the uncontrolled use of generative AI. Large language models can produce useful explanations and creative meal descriptions, but they can also invent ingredients, quantities, preparation steps, or nutrition claims. For a food-related application, this is not a minor issue. Incorrect allergen handling or fabricated nutritional information can damage user trust and may create health-related risk. The motivation of this project is therefore not to simply wrap an AI provider, but to place AI behind a deterministic validation and ranking process.

The fifth problem is collaborative household usage. Food inventory is often shared by multiple people, yet many applications model inventory as a private list owned by a single user. This does not reflect families, roommates, shared kitchens, or office environments. Shared inventory requires group membership, invitations, member-aware consumption, notifications, and stock deduction rules. Without these workflows, an inventory-aware recommendation system remains incomplete for real-world use.

The motivation behind this project is to build a single integrated platform that connects these concerns. The system aims to recommend meals from verified data, adapt results to user profiles, account for inventory availability, support collaborative inventory management, calculate nutrition from normalized quantities, and use AI only as an optional enhancement layer. This makes the project both a software engineering challenge and an applied recommendation-system problem. It requires backend architecture, data modeling, security, frontend usability, data processing, and quality assurance to work together rather than exist as separate demonstrations.

3. Project Scope and Objectives

The project scope covers the implementation of a full-stack meal recommendation and nutrition management platform rather than a standalone recommendation algorithm. The system was designed to support the complete user workflow around meal decisions: defining a personal profile, managing available ingredients, receiving recipe or menu suggestions, logging consumed meals, tracking nutritional impact, and maintaining the underlying recipe and ingredient data. This scope makes the project broader than a simple recipe search application, but still focused around one central question: how can software recommend meals that are personalized, practical, explainable, and technically reliable?

The first objective is to provide profile-based personalization. The platform stores and uses user information such as diet type, dietary goal, allergies, disliked ingredients, physical measurements, activity level, and daily calorie target. These values influence both filtering and ranking. Safety-related constraints, especially allergies and incompatible diet types, are treated as strict filters, while softer preferences such as disliked ingredients and cravings influence scoring and explanation.

The second objective is to make recommendations aware of the user’s real inventory. The platform supports inventory groups, inventory items, ingredient quantities, units, members, invitations, and shopping-list workflows. This allows recommendations and consumption actions to be connected to actual household food availability. Inventory is not modeled only as a private user list; it can represent shared locations such as a home, office, or common kitchen.

The third objective is to support AI-assisted recipe and menu recommendation without depending entirely on AI output. The backend first obtains safe recipe candidates, calculates nutrition where needed, scores recipes using deterministic factors, and only then allows an AI provider to generate explanations or structured menu selections. A free mode and fallback behavior keep the system usable when no external AI key is provided or when an AI service is unavailable.

The fourth objective is to provide nutrition and consumption tracking. Users can log consumed recipes, ingredients, inventory items, or custom food entries. The system supports nutrition preview, daily summaries, history views, and analysis over time. This connects recommendation with feedback: meals that users consume or mark as cooked can influence future recommendation diversity and usage-aware ranking.

The fifth objective is to manage recipes and ingredient data as first-class domain objects. The platform includes recipe browsing, creation, editing, image support, ratings, favorites, approval workflows, and version-related behavior. Ingredients include nutrition information and unit conversion data, allowing the system to calculate recipe-level nutritional values from ingredient-level records instead of relying only on manually entered totals.

The sixth objective is to provide administrative and operational support. Admin users can manage users, recipes, ingredients, approval workflows, and test inventory setup. The backend includes structured exception handling, localized messages, DTO mapping, security configuration, and database migrations. The project also includes Python utilities for cleaning, validating, and importing data into the relational model.

The seventh objective is maintainability. The system is organized into backend modules for infrastructure, domain, application, and utilities, while the frontend is organized around feature folders and shared infrastructure. This supports separation of concerns, easier testing, and future extension. The project therefore evaluates not only whether recommendations can be generated, but also whether the system can be maintained as a realistic software product.

In summary, the implemented scope includes authentication, profile management, inventory management, collaborative inventory workflows, recipe management, recommendation generation, menu construction, nutrition calculation, consumption tracking, notifications, data preparation, administration, and deployment support. Features outside the current scope include a native mobile application, production-scale collaborative filtering based on large user populations, and medical-grade dietary diagnosis. These are considered future extensions rather than required parts of the delivered graduation project.

4. Literature Background

Food recommendation systems have developed from simple search and filtering tools into systems that combine user profiles, recipe attributes, nutritional data, behavioral signals, and machine learning techniques. Traditional recommendation approaches include collaborative filtering, content-based filtering, and knowledge-based recommendation. Collaborative filtering can discover patterns from user interactions, but it is limited in cold-start situations and may struggle with strict dietary constraints. Content-based filtering can reason over recipe ingredients,

categories, and nutrition values, making it more suitable for explicit diet and allergy requirements. Knowledge-based systems encode domain rules directly, which is valuable in nutrition-related applications where safety and correctness matter [1], [2], [3].

Recent research emphasizes hybrid recommendation systems because food choice is a multi-objective problem. A useful food recommender must balance preference satisfaction, nutritional adequacy, ingredient availability, diversity, preparation practicality, and user trust. Hybrid systems combine the strengths of multiple approaches rather than relying on a single signal. This direction is directly relevant to the platform because the implemented recommendation strategy combines deterministic filtering, inventory matching, nutrition scoring, craving relevance, rating signals, recent usage history, and optional AI-generated explanation.

Ingredient-based recipe recommendation is especially important for practical home cooking. Several studies discuss recommendation based on available ingredients, partial ingredient matching, and missing-ingredient handling [4], [5]. This area motivates the project's inventory-aware design. Instead of recommending recipes only from abstract user preferences, the system considers what the user or household currently has. This supports practicality and can also contribute to food waste reduction by encouraging the use of available ingredients before additional shopping is needed.

Nutrition and diet planning literature shows that digital systems must connect recommendations with health goals, nutrient calculation, and monitoring over time [3], [6], [7], [8]. For this project, nutrition is not treated as an optional label attached to a recipe. It is calculated from ingredient-level nutritional records and recipe quantities, which requires unit conversion and normalized gram values. This is consistent with the broader view that diet planning systems should combine food databases, user goals, and tracking mechanisms rather than only recommend attractive meals.

Artificial intelligence and machine learning have become common in food and nutrition applications, including meal planning, recipe retrieval, dietary assessment, and personalized recommendation [5], [8]. Large language models can improve natural-language interaction and explanation quality, but the literature also highlights trust, reliability, and transparency as central adoption factors [9]. This project reflects that concern by limiting AI to an enhancement role. The AI provider receives a validated candidate pool and is expected to return structured output, while deterministic backend logic remains responsible for filtering, ranking, and fallback behavior.

Explainability is another recurring theme in AI-supported nutrition systems. Users are more likely to trust recommendations when they can understand why a meal was suggested: for example, because it uses available ingredients, avoids allergies, fits a dietary goal, or balances recent consumption [9]. The platform therefore includes explanation-oriented recommendation output rather than returning only recipe identifiers. Explanations are useful both for user trust and for debugging the recommendation behavior during development.

Personalization literature distinguishes between explicit preferences, implicit behavior, hard restrictions, and evolving user needs [1], [8], [9]. This distinction influenced the project's user model. Allergies and diet types are treated differently from cravings or disliked ingredients. Consumption history and cooked-recipe markers are considered behavioral signals, while ratings and comments provide explicit feedback. This layered model makes personalization more realistic than a single preference list.

The literature also supports the use of modern web architecture for recommendation platforms. Spring Boot is commonly used for RESTful backend systems with layered architecture, security, and database integration, while React supports component-based frontend development and responsive user interfaces [10], [11]. Relational databases remain appropriate for food recommendation systems that require strong relationships among users, recipes, ingredients, nutrition records, inventories, and consumption logs [12]. This supports the project's use of a modular Spring Boot backend, React frontend, and PostgreSQL persistence model.

Finally, sustainability and social impact are relevant to the project context. Ingredient-aware recommendation and inventory management can reduce unnecessary purchases and help users consume food they already have [13], [14]. While this project does not claim to solve food waste at population scale, its design aligns with the idea that recommendation systems can support more responsible consumption when they are connected to actual inventory and user behavior.

Overall, the literature suggests that an effective meal recommendation platform should be hybrid, nutrition-aware, explainable, personalized, and grounded in reliable data. The implemented platform follows this direction by combining rule-based filtering, deterministic ranking, AI-assisted explanation, relational persistence, user feedback, and inventory-aware workflows.

5. Requirements

The requirements of the platform were shaped by the goal of building a complete meal recommendation and nutrition management system. The system needed to support ordinary users who manage their own meals and inventory, shared household users who collaborate around the same stock, and administrators who maintain platform data. The requirements therefore cover recommendation behavior, profile management, inventory workflows, recipe management, consumption tracking, security, data quality, and maintainability.

5.1 User Roles

The platform supports three main usage perspectives:

- **Guest or unauthenticated visitor:** Can access public-facing application areas depending on frontend configuration, but cannot use protected profile, inventory, recommendation, or admin workflows without authentication.
- **Authenticated user:** Can manage a personal profile, maintain inventory groups, receive recommendations, log consumption, rate recipes, manage favorites, respond to invitations, and view notifications.
- **Administrator:** Can manage users, ingredients, recipes, approval flows, and administrative setup workflows. Admin access is protected through role-based authorization.

5.2 Functional Requirements

The user profile module shall allow users to create and update personal information used for recommendation and nutrition tracking. This includes diet type, dietary goal, allergies, disliked

ingredients, gender, age, height, weight, activity level, and daily calorie target. The system shall use this information to personalize recommendations and support nutrition summaries.

The authentication module shall protect user-specific and administrative actions. Authenticated requests shall be validated with JWT-based security, and administrator operations shall require an admin role. The system shall synchronize authenticated identity information with the local user database so that application data can be associated with the correct user record.

The inventory module shall allow users to create inventory groups, add and update items, select units, view ingredient conversions, invite members, remove members, consume stock, and generate shopping-list information. Inventory workflows shall support shared usage rather than only a single private user inventory.

The recipe module shall allow users to browse, inspect, create, update, submit, approve, reject, favorite, rate, and attach images to recipes according to the user's permissions. The system shall preserve recipe-related data such as ingredients, quantities, nutrition values, category, status, and version-related information.

The recommendation module shall generate recipe recommendations and menu recommendations. It shall consider user profile data, allergies, diet type, disliked ingredients, cravings, inventory ingredients, recipe nutrition, ratings, preparation practicality, and recent usage history. The system shall support a free deterministic mode and optional AI-assisted modes.

The AI integration module shall allow selected AI providers to be used for recommendation explanation or menu selection. User-provided API keys shall not be stored as plaintext in the frontend. The backend shall be able to decrypt provided keys only when needed for provider calls. If an AI provider fails or returns invalid output, the system shall fall back to deterministic recommendations.

The consumption module shall allow users to log meals from recipes, ingredients, inventory items, or custom entries. It shall support nutrition previews, daily summaries, history views, date-range analysis, and optional inventory deduction. Consumption data shall contribute to the user's longer-term nutrition and recommendation context.

The notification and invitation modules shall support shared inventory collaboration. Users shall be able to receive, accept, reject, or cancel inventory invitations where applicable. The system shall provide notification listing, unread count, read marking, and deletion workflows.

The data processing utilities shall support cleaning, validating, and importing ingredient, recipe, nutrition, recipe-ingredient, and unit conversion datasets. Import logic shall validate relationships before writing data and should avoid importing orphaned records when possible.

5.3 Non-Functional Requirements

The platform shall be maintainable. Backend code shall be organized around clear module boundaries and layered responsibilities. Frontend code shall be organized by features and shared infrastructure. Shared types, services, and contexts shall reduce duplicated logic.

The platform shall be secure. Authentication shall be delegated to a dedicated identity provider, protected API routes shall require valid tokens, administrative routes shall require role checks, and sensitive AI API keys shall be encrypted before storage on the client side.

The platform shall be reliable under external-service failure. Recommendation generation shall remain usable without an external AI provider. AI responses shall be parsed and validated before being accepted, and fallback recommendations shall be available when external providers are unavailable.

The platform shall preserve data consistency. PostgreSQL shall store structured domain data, and Flyway migration scripts shall manage schema evolution. Entity relationships, foreign keys, and service-level transaction boundaries shall protect user, inventory, recipe, recommendation, and consumption data.

The platform shall be testable. Domain services, repositories, controllers, mappers, utilities, and integration-oriented infrastructure shall be covered by automated tests where appropriate. Test infrastructure shall support realistic database, authentication, and storage behavior without depending only on shallow in-memory substitutes.

The platform shall be usable. The frontend shall provide clear workflows, form validation, loading states, error states, feedback messages, responsive layouts, and protected navigation. Recommendation output shall include explanations so that users can understand why a recipe or menu was suggested.

The platform shall be extensible. The architecture shall allow new AI providers, additional recommendation strategies, new recipe categories, expanded dataset imports, and future clients to be introduced without rewriting the core domain model.

6. System Architecture

The platform is implemented as a full-stack web system with a modular backend, a browser-based frontend, relational persistence, identity management, object storage, and containerized runtime services. The architecture was designed to keep business logic, delivery logic, infrastructure integrations, and user interface code separated while still remaining manageable for a graduation project.

At the highest level, users interact with a React single-page application. The frontend communicates with the backend through REST APIs. The backend handles authentication validation, business rules, recommendation workflows, persistence, file storage integration, and AI provider orchestration. PostgreSQL stores structured application data, Keycloak acts as the OAuth2/OIDC identity provider, and MinIO provides S3-compatible object storage for media-related workflows. Docker Compose is used to run the supporting services together in a local development and demonstration environment.

Table 6.1 summarizes the high-level runtime communication flow of the platform.

Layer or service	Main responsibility	Communication direction
User browser	Presents the application interface and sends user actions.	Interacts with the React frontend.
React and TypeScript frontend	Handles routing, forms, authentication state, UI feedback, and API requests.	Sends REST requests to the backend.
Spring Boot REST backend	Applies security validation, business rules, recommendation workflows, persistence operations, storage integration, and AI orchestration.	Coordinates all server-side services.
Keycloak	Provides OAuth2/OIDC authentication, JWT issuing, and role information.	Used by the backend for token validation and by the frontend for login.
PostgreSQL	Stores structured application data such as users, recipes, ingredients, inventories, recommendations, and consumption logs.	Accessed by the backend persistence layer.
MinIO object storage	Stores media-related files such as recipe and profile images.	Accessed by the backend storage integration.
External AI provider	Optionally generates explanations or menu selections from validated candidate data.	Called by the backend only when an AI mode is selected.

6.1 Backend Architecture

The backend follows a modular layered architecture. Instead of placing all code in a single application package, the system separates concerns into infrastructure, domain, application, and utility areas. This improves maintainability and makes it easier to reason about dependencies.

- **Infrastructure:** Contains external service integration concerns such as AI provider adapters, network clients, object storage integration, QueryDSL support, and test infrastructure.
- **Domain:** Contains core entities, repositories, services, recommendation strategies, inventory logic, recipe logic, consumption logic, user logic, and domain exceptions.
- **Application:** Contains the Spring Boot entry point, REST controllers, DTOs, mappers, security configuration, exception handling, localization configuration, and Flyway migrations.
- **Utilities:** Contains shared utility code and Python data-processing scripts used for dataset preparation.

This structure allows the core recommendation and nutrition logic to remain separate from HTTP delivery, authentication configuration, and external provider implementations. For example, AI

provider communication is treated as an external integration, while ranking and fallback behavior remain part of the application's business logic.

6.2 Frontend Architecture

The frontend is implemented with React, TypeScript, and Vite. It follows a feature-oriented structure rather than a page-only structure. Major features include dashboard, recommendations, inventory, consumption, recipes, profile, notifications, admin, about, and landing-page areas.

Shared frontend infrastructure includes authentication context, typed service modules, dependency injection utilities, theme context, toast feedback, reusable layout components, loading states, error states, empty states, and unit-converter components. This organization keeps feature code close to the UI workflows it supports while allowing common behavior to be reused across the application.

The frontend is intentionally kept as a client for structured backend APIs. It handles user interaction, form state, routing, feedback, and display logic, while business-critical calculations such as recommendation ranking, inventory deduction, calorie summaries, and unit conversion are handled by backend services.

6.3 Data and Persistence Architecture

PostgreSQL is used as the primary relational database. The data model includes users, recipes, ingredients, ingredient nutrition, recipe ingredients, inventory groups, inventory items, invitations, notifications, consumption logs, ratings, favorites, and recommendation history. Relational persistence is suitable for this domain because most workflows depend on strong relationships between users, ingredients, recipes, inventory groups, and consumption records.

Flyway is used to manage schema evolution. Instead of relying on manual database changes, structural changes are represented as versioned migration scripts. This makes the database state reproducible and helps keep development, testing, and deployment environments aligned.

6.4 Security Architecture

Authentication is delegated to Keycloak. The backend acts as an OAuth2 resource server and validates JWT tokens issued by Keycloak. Public routes are explicitly permitted, authenticated routes require a valid token, and administrative routes require an admin role. A custom role conversion layer maps Keycloak roles into Spring Security authorities.

The frontend integrates with the authentication layer through service abstractions. Protected routes prevent unauthenticated access to private workflows, while admin interfaces are guarded by role-aware checks. User-provided AI API keys are encrypted before being stored on the client side and decrypted by the backend only when an AI provider call is required.

6.5 External Integration Architecture

The system integrates with external AI providers through provider abstractions. The recommendation workflow can operate in a free deterministic mode without making an AI request. When an AI model is selected, the backend sends a curated and validated candidate set to

the selected provider. This isolates the system from direct dependence on AI output and allows graceful fallback behavior.

MinIO is used for S3-compatible object storage. This keeps binary file handling separate from relational data and supports recipe or profile image workflows without storing media directly inside PostgreSQL.

6.6 Deployment Topology

The local runtime topology is managed through Docker Compose. The Compose configuration defines services for PostgreSQL, Keycloak, MinIO, the Spring Boot backend, and the frontend served through Nginx. These services communicate over a dedicated Docker network. This arrangement reduces environment drift and allows the project to be demonstrated as a coordinated application stack rather than isolated components.

Overall, the architecture prioritizes separation of concerns, reliable data flow, secure identity handling, and controlled AI integration. It keeps the system practical for a university project while following patterns that are close to real-world full-stack application development.

7. Backend Design

The backend is the main decision-making layer of the platform. It is responsible for validating authenticated requests, applying domain rules, managing persistence, calculating nutrition, generating recommendations, coordinating external integrations, and returning stable DTO responses to the frontend. The design follows a modular layered structure so that HTTP concerns, domain behavior, persistence, security, and utility logic do not collapse into a single tightly coupled application layer.

7.1 Module Organization

The backend is organized into separate Gradle modules. This structure creates clearer dependency boundaries and makes the codebase easier to understand as the project grows.

- **01-infrastructure:** Contains concrete infrastructure concerns such as external AI provider implementations, network clients, MinIO storage integration, QueryDSL-related support, and shared test infrastructure.
- **02-domain:** Contains core domain entities, repositories, services, recommendation strategies, inventory logic, recipe logic, consumption logic, notification logic, user logic, and custom domain exceptions.
- **03-application:** Contains the Spring Boot application entry point, REST controllers, application services, DTOs, MapStruct mappers, security configuration, localization configuration, global exception handling, and Flyway migration scripts.
- **04-utilities:** Contains reusable utility logic such as encryption helpers, calorie calculation support, and Python-based data processing scripts.

This organization allows the backend to separate technical integration details from business behavior. For example, the recommendation strategy can depend on a prompt engine abstraction

while the infrastructure module provides concrete AI provider clients. Similarly, application controllers do not directly implement recommendation scoring; they delegate to application and domain services.

7.2 Application Layer and REST Controllers

The application layer exposes the backend through REST controllers. Controllers are organized around domain capabilities such as users, recipes, ingredients, inventory groups, inventory invitations, notifications, consumption, ratings, recommendations, definitions, and administrative operations.

Controllers are intentionally kept close to request handling responsibilities. Their main tasks are to receive HTTP requests, validate request bodies where applicable, extract authenticated identity information, call the appropriate service, and return DTO responses. Business workflows such as recommendation generation, inventory deduction, calorie calculation, and recipe approval are handled below the controller layer.

This separation keeps the HTTP layer easier to test and prevents controller classes from becoming the location of core business rules. It also allows future clients, such as a mobile application, to consume the same backend behavior through the same REST contracts.

7.3 Domain Services

Domain services implement the core business behavior of the platform. Important examples include:

- User profile synchronization, preference management, and identity relinking behavior.
- Inventory group management, member handling, item updates, stock consumption, and shopping-list generation.
- Recipe lifecycle behavior, including recipe creation, updates, approval flow, version-related behavior, ratings, favorites, and image-related metadata.
- Recommendation generation for individual recipes and menus.
- Daily consumption logging, nutrition preview, history, and analysis.
- Ingredient lookup, nutrition handling, and unit conversion support.

The recommendation-related services are especially central to the backend. The recipe recommendation flow obtains safe candidates, calculates nutrition, ranks candidates, optionally calls an AI provider, validates AI output, and stores recommendation history. Menu recommendation similarly builds category-based candidate pools and can produce deterministic fallback menus when an AI provider is not used.

7.4 Persistence and Repository Layer

The backend uses Spring Data JPA repositories for persistence. Repository interfaces provide access to users, recipes, ingredients, inventory groups, invitations, notifications, consumption records, ratings, favorites, recommendations, and related entities. QueryDSL support is used where more complex dynamic filtering is needed, especially around recipe and recommendation candidate selection.

The relational model is important for this system because many operations depend on joining user profile data, recipe data, ingredient data, inventory state, and consumption history. Repository methods are therefore not only CRUD accessors; they also support domain-specific queries such as safe recipe candidate retrieval, group membership checks, recommendation history lookup, and user search.

7.5 DTOs and Mapping

The backend separates internal domain entities from API response models. DTOs are used to define request and response shapes for frontend communication, while mapper classes convert between domain objects and API-facing models.

This design reduces coupling between persistence entities and external API contracts. If an entity changes internally, the API does not automatically have to expose every field. Likewise, the frontend receives data shaped for its workflows rather than raw database entities. This is particularly important for recommendation, menu, inventory, user, and recipe responses where nested data must be controlled.

7.6 Transaction Boundaries

Service methods use transaction boundaries for workflows that must update several related records consistently. Examples include user profile updates, identity relinking, recommendation persistence, rating updates, marking recommendations as cooked, logging consumption, and inventory deduction.

Transaction management is especially important in shared inventory and consumption workflows. A consumption action may create a consumption record and optionally deduct stock from an inventory group. These actions must either complete together or fail together so that the user's nutrition history and inventory state do not diverge.

7.7 Error Handling and Localization

The backend includes structured exception handling so that domain and validation errors can be returned to the frontend in a consistent format. Custom exceptions are used for common domain failures such as missing resources, insufficient stock, or invalid operations. This makes error behavior easier to understand and easier for the frontend to display.

Localization support is also present through message resources. This allows backend-generated messages, validation responses, or notification-related text to be adapted for multiple languages where needed. Since the frontend also uses an internationalization layer, the system can support a more consistent multilingual user experience.

7.8 Backend Design Rationale

The backend design prioritizes correctness and maintainability over placing all logic in the fastest possible path. This is appropriate for the project because recommendation quality depends on several coordinated concerns: secure identity, reliable data, valid nutrition calculation, safe

filtering, explainable ranking, and graceful AI fallback. By structuring the backend into modules and layers, the implementation remains easier to inspect, test, and extend.

In this design, AI integration is intentionally not the center of the backend. The core backend remains capable of producing recommendations without external AI. This protects the platform from provider failures and makes the recommendation process more trustworthy, since data validation and ranking happen inside the controlled application domain.

8. Database Design and Flyway Evolution

The platform uses PostgreSQL as its primary relational database. A relational design is appropriate for this project because the domain depends heavily on structured relationships: users have profiles and preferences, recipes contain ingredients, ingredients have nutrition records and unit conversions, inventories belong to groups, groups have members, recommendations contain selected recipes, and consumption records connect users with meals over time.

The database is not treated as a passive storage layer. It is part of the correctness model of the system. Foreign keys, join tables, entity relationships, and transactional service methods help preserve consistency between user profiles, inventory state, recommendation history, and consumption logs. This is important because recommendation quality depends on trustworthy data.

8.1 Core Data Areas

The schema can be understood through several major data areas:

- **User and profile data:** User records store identity-linked profile information such as email, name, dietary goal, diet type, allergies, disliked ingredients, physical measurements, activity level, role, active status, and daily calorie target.
- **Recipe and ingredient data:** Recipe records store title, instructions, category, status, image metadata, preparation information, rating-related fields, and version-related relationships. Ingredient records store names, categories, physical state, density, nutrition data, and unit conversion records.
- **Inventory data:** Inventory groups represent shared locations, while inventory items connect groups to ingredients, quantities, and units. Membership and invitation data support collaborative usage.
- **Consumption data:** Daily consumption records preserve logged meals, nutrition estimates, meal type, portion information, timestamps, and links to recipes where applicable.
- **Recommendation data:** Recommendation sessions and recommended recipe records preserve generated outputs, AI model context, explanation text, user feedback, cooked markers, and usage information.
- **Notification data:** Notification records support user-facing events such as inventory invitations and workflow updates.

8.2 Important Relationships

The most important relationships in the schema are:

- A user can belong to multiple inventory groups, and an inventory group can contain multiple users.
- An inventory group contains inventory items, each connected to an ingredient.
- A recipe contains recipe-ingredient rows, each connected to an ingredient and a resolved quantity.
- Ingredient nutrition records provide macro values used to calculate recipe nutrition.
- Ingredient unit records provide gram conversion data for practical cooking units.
- A user can create ratings, favorites, recommendations, and consumption logs.
- A recommendation session can contain multiple recommended recipes.
- Notifications and invitations connect collaboration events back to users and inventory groups.

These relationships allow the system to answer practical questions such as: which recipes are safe for this user, which recipes match the current inventory, which ingredients are missing, how many calories a recipe contributes, and how recent consumption should affect future recommendations.

8.3 Flyway Migration Strategy

Database schema evolution is managed through Flyway. In the current repository state, the migration directory contains versioned scripts from V1 through V29. This means the schema can be recreated and evolved in a predictable order when the application starts in a new environment.

Flyway provides several advantages for this project:

- It prevents uncontrolled manual database changes.
- It makes schema history visible in the repository.
- It allows new environments to be initialized consistently.
- It supports incremental changes as the project scope grows.
- It reduces the risk of development and demonstration environments drifting apart.

8.4 Migration Milestones

The migration history reflects the evolution of the project from a basic recipe and nutrition model into a broader meal planning platform.

Migration range	Main purpose
V1-V3	Initial schema setup and early user identity adjustments.

Migration range	Main purpose
V4-V10	Inventory groups, disliked ingredients, smart consumption fields, ingredient units, density, physical state, and shared inventory migration.
V11-V12	Inventory invitations and notification support.
V13-V16	User BMI-related data, recipe ingredient amount/unit fields, user roles, active flags, recipe images, and missing recipe fields.
V17-V21	Recipe status, creator data, status logic updates, parent recipe relationships, favorites, and recipe categories.
V22-V25	Preferred ingredient units, liquid density fixes, comprehensive unit/density improvements, and recipe ingredient constraint changes.
V26-V29	Recommendation tables, recommendation usage tracking, recipe version numbers, and recipe diet type support.

This progression shows that the schema was extended as the product scope matured. Inventory collaboration, recipe lifecycle management, recommendation persistence, unit conversion, and diet-aware recipe data were added gradually rather than being forced into the initial schema.

8.5 Schema Design Rationale

The database design favors normalized relational structures over storing large unstructured documents. This makes sense for the system because nutrition calculation, inventory matching, allergy filtering, and recommendation ranking require precise relationships between users, recipes, ingredients, and quantities.

For example, recipe nutrition is calculated from recipe ingredients and ingredient nutrition values. This requires recipes, ingredients, recipe-ingredient links, and nutrition records to remain queryable as structured data. Similarly, inventory-aware recommendation requires inventory items to be connected to ingredients, not stored only as plain text.

The schema also supports future extension. Recommendation history can be used later for more advanced personalization. Consumption records can support trend analysis. Recipe versioning and approval fields allow safer recipe lifecycle management. Ingredient units and density values allow practical cooking measurements to be converted into gram-based calculations.

8.6 Data Consistency Considerations

Several workflows require consistency across multiple tables:

- Logging consumption may also deduct inventory stock.
- Accepting an inventory invitation affects group membership and notification state.

- Rating a recommended recipe can affect both recommendation feedback and recipe rating data.
- Updating a user identity may require preserving existing records under a new authentication subject.
- Approving or editing recipes affects visibility and version-related behavior.

For these cases, service-level transaction boundaries are important. The database design and backend transaction model work together so that related state changes do not partially complete in a way that would confuse users or weaken recommendation accuracy.

Overall, the database design supports the platform's main goal: producing reliable, personalized, inventory-aware, and nutrition-aware recommendations from structured data rather than from unverified text alone.

9. Authentication and Authorization

Authentication and authorization are central to the platform because most workflows are user-specific. Profile data, inventory groups, consumption logs, recommendation history, ratings, and notifications must be associated with the correct authenticated user. Administrative operations must also be separated from ordinary user workflows. For these reasons, the platform delegates identity management to Keycloak and uses Spring Security on the backend to validate requests and enforce access rules.

9.1 Keycloak-Based Authentication

Keycloak is used as the OAuth2/OIDC identity provider. Users authenticate through Keycloak rather than through a custom password system inside the application database. This design avoids storing user passwords in the meal recommendation backend and allows authentication concerns to remain separated from domain logic.

After authentication, Keycloak issues a JWT. The frontend attaches this token to API requests, and the backend validates it as an OAuth2 resource server. The token subject is used as the user's identity anchor in the application database.

9.2 Backend Resource Server Configuration

The backend configures Spring Security to define public, authenticated, and admin-only routes. Public endpoints include selected test, storage, AI test, definition, Swagger, and public paths. Administrative API paths require the ADMIN role, while other routes require an authenticated user.

This configuration makes the backend stateless from an HTTP session perspective. Each protected request must carry a valid JWT, and Spring Security validates the token before the request reaches protected business logic.

9.3 JWT Validation

The backend uses the configured issuer URI and JWK set URI to validate JWTs. The JWK set allows the backend to verify tokens signed by Keycloak, while issuer validation ensures that tokens come from the expected realm. Timestamp validation rejects expired or otherwise invalid tokens.

This approach is more reliable than accepting token contents blindly. It ensures that role and user information used by the backend are extracted from validated authentication data.

9.4 Role Mapping and User Synchronization

Keycloak roles are converted into Spring Security authorities using a custom converter. The converter reads realm_access.roles, prefixes each role with ROLE_, and returns a JwtAuthenticationToken. The same converter also synchronizes the authenticated user with the local database.

backend/modules/03-application/src/main/java/com/mealapp/app/config/KeycloakRoleConverter.java:

```
Collection<GrantedAuthority> authorities = Stream.concat(
    defaultAuthoritiesConverter.convert(jwt).stream(),
    roles.stream()
        .map(roleName -> "ROLE_" + roleName.toUpperCase())
        .map(SimpleGrantedAuthority::new)
).collect(Collectors.toSet());

userService.syncUser(new UserSyncRequest(
    jwt.getSubject(),
    jwt.getClaimAsString("email"),
    jwt.getClaimAsString("name"),
    List.copyOf(roles)
));

return new JwtAuthenticationToken(jwt, authorities, jwt.getClaimAsString("preferred_username"));
```

This design keeps the local user table aligned with authenticated Keycloak users. The application can therefore attach domain data such as profile settings, inventory groups, recommendations, and consumption records to the local user record while still relying on Keycloak for identity.

9.5 Identity Relinking

The platform includes identity relinking support for cases where a Keycloak subject changes but the same real user should keep existing application data. This can happen during local development, realm recreation, migration, or identity-provider changes. Without relinking, records connected to the old subject could become disconnected from the user.

The backend checks whether a user with the authenticated subject already exists. If not, it can attempt to find an existing local user by the authenticated email address and relink the stored user ID to the new subject. The database migration V3__allow_user_id_relink.sql also updates

selected user foreign key constraints with ON UPDATE CASCADE, allowing related rows to follow the user ID change.

backend/modules/03-application/src/main/java/com/mealapp/app/controller/UserController.java:

```
User existingByEmail = userService.findByEmail(authenticatedEmail).orElse(null);
if (existingByEmail != null) {
    log.info("Relinking user profile from legacy Keycloak subject {} to {}",
        existingByEmail.getId(), authenticatedUserId);
    userService.relinkUserId(existingByEmail.getId(), authenticatedUserId);

    User relinkedUser = userService.findById(authenticatedUserId)
        .orElseThrow(() -> new ResourceNotFoundException(
            "Kullanici relink isleminden sonra bulunamadi ID: " + authenticatedUserId));

    updateUserFields(relinkedUser, request, authenticatedEmail);
    User saved = userService.save(relinkedUser);
    return userMapper.toDto(saved);
}
```

This mechanism is especially useful in a project environment where authentication realms may be recreated during development and testing. It protects user-associated application data from being lost only because an external identity subject changed.

9.6 Frontend Authentication Flow

The frontend uses an authentication context and authentication service abstraction. This keeps React components from depending directly on low-level Keycloak implementation details. The provider tracks initialization state, authentication state, the current user, login, registration, logout, and authentication errors.

This enables route guards, navigation components, and feature screens to consume authentication state consistently. It also allows the project to support different authentication service implementations where needed, such as a guest-mode implementation for local frontend operation.

9.7 Authorization Model

The authorization model is role-based at the API boundary and ownership-aware inside services. Admin routes require an admin role. Authenticated user routes require a valid token. For resource-specific behavior, service-layer checks are needed to ensure that users can only modify data they are allowed to access, such as their own profile or inventory groups where they are members.

This layered authorization model is important because not every permission can be expressed only through URL patterns. For example, two authenticated users may call the same inventory endpoint, but only members of a specific group should be allowed to modify that group’s items.

9.8 Security Rationale

The security design follows three principles:

- **Do not store passwords in the application backend.** Authentication is delegated to Keycloak.
- **Validate every protected request.** JWT validation is handled by Spring Security using issuer and JWK validation.
- **Keep authorization close to the protected resource.** URL-level role checks protect broad areas, while service-level checks protect ownership and membership behavior.

This design gives the platform a realistic security foundation for a full-stack application while keeping authentication infrastructure separate from recommendation, inventory, and nutrition domain logic.

10. AI-Powered Recommendation System

The recommendation system is one of the central components of the platform. Its purpose is to suggest recipes or menus that are safe, personalized, practical, and explainable. The project does not rely on generative AI as the only decision maker. Instead, it uses a hybrid architecture where deterministic backend logic filters and ranks candidate recipes before any optional AI provider is used.

This design choice is important for a food-related system. A meal recommendation may involve allergies, diet types, nutrition goals, ingredient availability, and user trust. If an AI model is allowed to invent recipes or nutrition values without validation, the output may become unsafe or unusable. The platform therefore treats AI as an enhancement layer that can produce explanations or structured selections from a controlled candidate pool.

10.1 Recommendation Flow

The recipe recommendation flow follows these major steps:

1. The authenticated user’s profile and recommendation request are loaded.
2. The system collects the current inventory context and daily nutrition summary.
3. Recipes are filtered against hard constraints such as diet type and allergies.
4. Candidate recipes are scored using deterministic ranking factors.
5. If the selected model is FREE, the backend returns deterministic recommendations directly.
6. If an AI provider is selected, the backend sends structured candidate data to the provider.
7. The AI response is parsed as structured JSON and matched back to known recipes.
8. If the AI response is empty, invalid, or unusable, deterministic fallback recommendations are returned.
9. The recommendation session and selected recipes are persisted for history and future feedback.

This flow keeps the system functional even when no AI provider is configured. It also prevents the AI layer from selecting recipes that are not present in the validated candidate set.

10.2 Deterministic Candidate Filtering

The first stage of recommendation is safety-oriented filtering. The backend retrieves recipes that are compatible with the user's diet type and allergies. This step is not delegated to AI because it represents a hard correctness requirement. Recipes that violate allergies or incompatible diet rules should not reach later ranking stages.

After safe candidates are retrieved, the system calculates nutrition data for candidates where needed. This ensures that nutrition proximity can be considered as part of ranking and explanation.

10.3 Recipe Ranking Factors

The recipe ranking strategy combines several signals:

- **Inventory match:** Rewards recipes that can be prepared with available ingredients.
- **Taste preference:** Penalizes recipes containing disliked ingredients.
- **Craving match:** Rewards recipes related to the user's current craving text.
- **Nutrition match:** Scores recipes against the user's calorie and dietary context.
- **Rating:** Uses community or user rating signals where available.
- **Cook history:** Considers previous usage so repeated meals can be handled differently.
- **Preparation time:** Rewards practical recipes with shorter preparation times.
- **Diversity:** Penalizes recently consumed or recently recommended recipes.

These weights show that the system gives the strongest emphasis to inventory fit while still considering preference, craving, nutrition, diversity, and practical cooking factors.

10.4 Free Mode and Fallback Behavior

The FREE model is a non-AI mode. When the user selects it, the backend skips the external AI call and returns fallback recommendations built from the deterministic ranking result. This is important for accessibility because users can still use the recommendation system without providing an API key.

Fallback behavior is also used when an AI provider returns an empty response, invalid JSON, or a response that cannot be matched to known recipes. This makes the recommendation workflow resilient to external-provider failures.

10.5 AI Response Validation

When an AI provider is used, the backend expects structured output. The raw response is sanitized, parsed, and matched back to the ranked recipe list. If parsing fails, the backend does not expose the invalid AI result to the user. It falls back to deterministic recommendations.

This validation step is one of the main safeguards against hallucinated or malformed AI responses. The AI layer can improve explanation quality, but it cannot bypass backend validation.

10.6 Menu Recommendation

In addition to individual recipe recommendation, the system supports menu recommendation. Users can select recipe categories such as soups, main dishes, or desserts. The backend builds candidate pools for each category, scores candidates, and constructs menu alternatives.

Menu recommendation uses a separate scoring strategy focused on:

- Inventory match
- Palate and craving relevance
- Nutrition match
- Diversity
- Popularity and preparation practicality

If the selected model is free or if AI menu generation fails, the backend builds fallback menus from the ranked candidate pools. This keeps menu recommendation available even without external AI.

10.7 AI Provider Integration

AI providers are integrated through provider abstractions. The frontend exposes FREE, GEMINI, OPENAI, and CLAUDE as selectable options. The backend infrastructure includes provider implementations for OpenAI, Gemini, Claude, and a generic OpenAI-compatible provider that can be configured through application settings.

This design allows the system to route requests to the selected provider while preserving a configured default provider. It also makes provider replacement or extension easier because new providers can implement the same provider contract.

10.8 API Key Handling

For paid AI providers, users can provide their own API keys. The frontend encrypts keys before storing them locally, and the backend decrypts provided keys only when a recommendation request requires an external provider call. The AI recommendation flow therefore supports bring-your-own-key usage without storing plaintext keys in browser storage.

This feature improves flexibility because the system can be used in free deterministic mode or with user-selected AI providers. It also keeps AI-provider credentials separate from the main application authentication system.

10.9 Explainability

The recommendation output includes explanation text. In AI-assisted mode, the explanation can be produced by the selected provider from validated candidate data. In deterministic fallback mode, backend-generated insights explain the recommendation using factors such as matched inventory ingredients, missing ingredients, diet compatibility, nutrition fit, ratings, preparation time, and craving relevance.

Explainability is important because users are more likely to trust a recommendation when they can understand its basis. In this project, explanation is not only a cosmetic feature. It is part of the system's trust model and helps distinguish validated recommendations from arbitrary AI-generated meal ideas.

10.10 Design Rationale

The recommendation design follows a controlled-AI pattern:

- The database defines the available recipes and ingredients.
- The backend filters unsafe candidates.
- The backend ranks candidates using deterministic factors.
- AI is optional and receives only curated candidate data.
- AI output is parsed and validated before use.
- Fallback recommendations are always available.

This architecture balances the strengths of deterministic recommendation and generative AI. Deterministic logic provides safety, repeatability, and fallback reliability. AI improves explanation and selection flexibility when available. Together, they form a recommendation system that is practical for users and maintainable for developers.

11. Nutrition, Calorie, and Unit Conversion Methodology

Nutrition handling is central to the platform because meal recommendation is not limited to finding recipes that look relevant. The system also reasons about profile-based calorie targets, recipe nutrition, ingredient quantities, practical household units, and nutrition-aware recommendation scoring. These calculations are implemented on the backend rather than duplicated across frontend components.

The methodology combines four concerns: estimating a user's daily calorie target, calculating recipe and serving-level nutrition, converting practical ingredient units into gram-based quantities, and using nutrition proximity as one of the recommendation signals.

11.1 Nutrition Calculation Strategy

Concern	Methodology	Design reason
Daily calorie target	Uses profile data such as weight, height, age, gender, activity level, and dietary goal when available. Missing data falls back to a conservative default.	Keeps recommendation usable even when profiles are incomplete.
BMI and profile indicators	Derives BMI when height and weight are available and presents it as contextual information, not as a complete health assessment.	Avoids overstating medical precision while still supporting useful profile context.
Recipe nutrition	Calculates total and per-serving macro values from recipe ingredients, serving count, and ingredient-level nutrition records.	Prevents the frontend from manually estimating recipe nutrition.
Nutrition proximity	Compares estimated recipe calories with the user's daily target as one ranking signal among several.	Helps rank meals by suitability without making nutrition the only decision factor.
Missing data handling	Uses safe fallbacks, neutral scoring, and defensive serving-count behavior.	Prevents imported or incomplete data from breaking recommendation workflows.

This strategy provides repeatable estimates suitable for meal planning, but it is not presented as medical-grade dietary advice. The system supports nutrition awareness, not clinical diagnosis or prescription.

11.2 Ingredient-Level Unit Conversion

A major challenge in recipe and inventory systems is unit consistency. Users and datasets often describe ingredients with practical units such as cups, tablespoons, teaspoons, pieces, slices, cloves, or packages. Nutrition calculations, however, are most reliable when quantities can be converted to grams. The unit conversion service therefore converts ingredient quantities into gram-equivalent values before nutrition, inventory, and consumption logic depend on them.

backend/modules/03-application/src/main/java/com/mealapp/app/service/impl/UnitConverterServiceImpl.java:

```
public double convertToGrams(double amount, String unit, Ingredient ingredient) {
    return amount * getUnitGramWeight(unit, ingredient);
}

private double getUnitGramWeight(String unit, Ingredient ingredient) {
    String normalizedUnit = normalizeUnit(unit);
    if (normalizedUnit == null || normalizedUnit.isBlank()) {
        return DEFAULT_GRAMS;
    }

    Double ingredientSpecific = getIngredientSpecificUnitWeight(normalizedUnit, ingredient);
    if (ingredientSpecific != null) {
        return ingredientSpecific;
    }

    String canonical = UNIT_ALIASES.get(normalizedUnit);
    if (canonical != null) {
        Double aliasSpecific = getIngredientSpecificUnitWeight(canonical, ingredient);
        if (aliasSpecific != null) {
            return aliasSpecific;
        }
        return STANDARD_UNIT_GRAMS.getOrDefault(canonical, DEFAULT_GRAMS);
    }

    return STANDARD_UNIT_GRAMS.getOrDefault(normalizedUnit, DEFAULT_GRAMS);
}
```

The conversion order is important. The service first checks ingredient-specific weights, then alias mappings, and finally standard fallback values. This allows the system to treat one cup of a dense ingredient differently from one cup of a lighter ingredient when detailed unit data exists.

11.3 Practical Input Handling and Integration

Unit names can vary by dataset, user input, or language. The converter therefore normalizes unit text by trimming whitespace, lowercasing, removing punctuation, and converting Turkish characters into ASCII equivalents. This improves matching reliability for Turkish and mixed-format unit data.

Nutrition and unit conversion support several workflows at once. During recommendation, calorie estimates and nutrition proximity contribute to scoring. During inventory and consumption, gram-equivalent quantities make recipe requirements comparable with household stock. During profile personalization, calculated calorie targets provide context for ranking meals. Centralizing this logic in backend services keeps behavior testable, reusable, and consistent across the application.

Overall, nutrition and unit conversion are treated as core domain concerns. They support personalization, explainability, and inventory-aware recommendation while remaining transparent about approximation and data-quality limits.

12. Inventory and Shared Household Management

Inventory management is one of the main features that differentiates the platform from a simple recipe browser. The system does not only recommend meals from a static catalog; it also considers what users already have available. This makes recommendations more practical, supports household planning, and can reduce unnecessary shopping.

The inventory model is based on inventory groups rather than only private user-owned lists. A group represents a shared stock location such as a home kitchen, refrigerator, pantry, or common household inventory. Users can participate in multiple groups, and the same group can be shared through invitations.

12.1 Inventory Model and Core Workflows

Area	Design summary
Inventory groups	Represent shared stock locations and connect multiple users to the same inventory context.
Inventory items	Link an ingredient to a group with quantity and unit information. A uniqueness rule prevents duplicate ingredient rows inside one group.
Default group creation	New users can receive a default Home group so inventory-dependent workflows have a usable starting point.
Group access	Group operations are scoped by authenticated user membership, preventing users from managing groups they do not belong to.
Stock updates	Add, subtract, and set operations allow grocery entry, correction, and consumption workflows to share the same model.
Unit conversion	Stock operations can use the unit conversion service so household units and standardized quantities remain consistent.
Invitations	Users can invite others by email, and membership changes only after a pending invitation is accepted by the intended user.
Consumption deduction	Recipe or ingredient consumption can deduct stock from a selected group when the user chooses inventory as the source.
Shopping list	Low or missing inventory items can be converted into shopping-list responses for planning.

This model makes inventory a shared domain capability rather than a secondary CRUD list. Recommendation, consumption logging, and shopping planning all depend on the same inventory state.

12.2 Database Evolution Toward Shared Inventory

The Flyway migration history shows that inventory evolved from user-specific ownership into group-based shared inventory. A representative migration introduced the user_inventory_groups join table, migrated existing ownership data into the new relationship, and removed direct user ownership from inventory groups and inventory rows.

Representative migration excerpt from backend/modules/03-application/src/main/resources/db/migration/V10__migrate_to_shared_inventory.sql:

```
CREATE TABLE user_inventory_groups (
    user_id VARCHAR(255) NOT NULL,
    inventory_group_id BIGINT NOT NULL,
    PRIMARY KEY (user_id, inventory_group_id),
    CONSTRAINT fk_user_inventory_groups_user
        FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
    CONSTRAINT fk_user_inventory_groups_group
        FOREIGN KEY (inventory_group_id) REFERENCES inventory_groups(id) ON DELETE CASCADE
);

INSERT INTO user_inventory_groups (user_id, inventory_group_id)
SELECT user_id, id FROM inventory_groups;

ALTER TABLE inventory_groups DROP COLUMN user_id;
ALTER TABLE inventories DROP COLUMN user_id;
```

This change is significant because the access boundary becomes group membership. Inventory items belong to groups, users belong to groups, and backend services can authorize inventory operations through membership checks.

12.3 Shared Usage, Safety, and Frontend Support

The inventory module supports collaborative household usage through invitations, member management, and group-scoped stock operations. When users log consumption from inventory, the backend can deduct ingredient quantities from the selected group. Before deduction, the service checks available stock and raises a domain exception if the quantity is insufficient. This prevents silent negative inventory and keeps stock data trustworthy.

On the frontend, inventory functionality is exposed through typed service functions and a dedicated inventory hook. The hook coordinates selected group state, item loading, invitation modals, conversion previews, member search, shopping-list loading, and stock consumption actions. This keeps the UI workflow manageable while the backend remains responsible for authorization, conversion, and consistency.

Overall, the inventory design supports the central project goal: recommendations are grounded in the user's real kitchen context. It also connects household collaboration, unit conversion, consumption logging, and shopping-list generation into one coherent workflow.

13. Recipe Lifecycle and Approval Workflow

Recipes are primary domain objects in the platform. They support browsing, recommendation, consumption logging, nutrition calculation, favorites, ratings, and menu generation. Because recipes influence many workflows, the system does not treat recipe creation as an immediate public insert. Instead, it uses a controlled lifecycle with drafts, pending submissions, administrative moderation, versioned updates, and visibility rules.

The lifecycle has two goals. First, users can create and edit recipes without losing work. Second, the public recipe catalog remains moderated so that recommendation and nutrition workflows do not depend on unreviewed content.

13.1 Recipe Model and Lifecycle States

The recipe entity stores both content and structured recommendation data: title, ingredients, category, diet type, instructions, preparation time, serving count, difficulty, rating data, nutrition totals, image URL, publication status, creator, parent recipe reference, and version number. The parentId and versionNumber fields support revision handling by keeping an approved public recipe stable while a proposed update is reviewed separately.

State or concept	Meaning in the workflow
DRAFT	A private recipe saved by the creator before submission.
PENDING_APPROVAL	A recipe or revision waiting for administrative review.
APPROVED	A public recipe that can be shown to users and used in recommendation workflows.
REJECTED	A submitted recipe or revision that was not approved but can remain visible to the creator for correction.
SUPERSEDED	A revision that has been replaced by a newer draft, pending revision, or approved update.
parentId	Connects a proposed update back to the original approved recipe.
versionNumber	Preserves the order of recipe revisions.

This state model makes moderation explicit. A recipe is not simply present or absent; its lifecycle state determines visibility, editing rights, and whether it can be used as public recommendation content.

13.2 User and Administrator Workflow

Workflow step	System behavior
Create recipe	The backend records the authenticated creator, saves recipe ingredients, calculates nutrition totals, and stores the recipe as a draft or pending submission.
Save draft	The user can keep incomplete work private and continue editing later.
Submit for approval	Only valid draft or rejected recipes can be submitted. Duplicate pending submissions are rejected.
Admin review	Administrators can list pending recipes and approve or reject them through protected endpoints.
Approve new recipe	The pending recipe becomes public by changing its status to approved.
Approve update	The pending revision is copied into the original approved recipe, and the temporary revision is marked inactive or superseded.
Reject submission	The status becomes rejected, preserving the creator’s ability to revise and resubmit.
Edit approved recipe	The backend creates a separate revision rather than overwriting the public record directly.
Upload image	Draft images can be updated directly; approved recipe image changes follow the same moderated revision model.

This workflow protects public catalog quality while still allowing user contribution. Regular users can create, edit, save, and submit recipes, while administrators control which content becomes public.

13.3 Visibility, Nutrition, and Design Rationale

Recipe visibility depends on status, activity, ownership, and administrator privileges. Approved root recipes are visible to normal users. Drafts, pending revisions, rejected recipes, and superseded records are restricted according to ownership and moderation rules. This prevents private or unreviewed content from entering recommendation results before review.

Recipe lifecycle management is also connected to nutrition calculation. When recipes are created or updated, the backend calculates calories, protein, carbohydrates, and fat from ingredient-level nutrition and gram quantities. If ingredient grams are missing, the unit converter can resolve them from amount and unit. This ensures that approved recipes and pending revisions carry structured nutrition data for recommendation and consumption workflows.

Overall, the recipe lifecycle keeps the catalog reliable without blocking user participation. Approved recipes remain stable, proposed updates are isolated until review, nutrition values are recalculated when ingredients change, and visibility rules protect private drafts and pending revisions.

14. Data Engineering and Dataset Preparation

The platform depends on structured recipe, ingredient, nutrition, and unit conversion data. Without a consistent dataset, the recommendation engine cannot reliably compare recipes with inventory, calculate nutrition values, or support household units. For this reason, the project includes Python-based data-processing utilities under backend/modules/04-utilities/data-processor/python/.

The data pipeline prepares Excel-based data for PostgreSQL import and aligns it with the backend domain model used by ingredients, ingredient nutrition, ingredient units, recipes, and recipe ingredients.

14.1 Pipeline Goals and Processing Stages

The pipeline has four main goals: clean ingredient names, reduce safe duplicate records, standardize recipe fields, and import relational data without breaking references. Table 14.1 summarizes the main processing stages.

Stage	Purpose	Main implementation behavior
Text normalization	Reduce spelling and formatting inconsistency.	Normalizes ingredient names, applies known typo fixes, trims values, and prepares enum-style recipe fields.
Duplicate handling	Merge only highly similar ingredient records.	Uses conservative fuzzy matching and remaps dependent recipe, nutrition, and unit rows to cleaned ingredient ids.
Unit and density enrichment	Support practical household units.	Assigns physical state, preferred unit, and density defaults where source data is incomplete.
Workbook output	Produce inspectable intermediate data.	Writes cleaned sheets for ingredients, recipes, recipe ingredients, ingredient units, and ingredient nutrition.

Stage	Purpose	Main implementation behavior
Preflight validation	Detect relational problems before database writes.	Checks for missing ingredient or recipe references in nutrition, unit, and recipe-ingredient rows.
Numeric normalization	Prevent spreadsheet formatting issues.	Converts comma decimal values and maps source nutrition columns to backend database columns.
Transactional import	Keep database state coherent.	Imports cleaned data inside a transaction and rolls back on SQL failure.

This design treats dataset preparation as a relational transformation rather than only a string-cleaning task. When ingredient identifiers change, dependent tables are remapped so that recipe and nutrition relationships remain valid.

14.2 Import Safety and Runtime Integration

The importer supports a dry-run mode so developers can inspect problems before making database changes. This is important because the import process can truncate and reload core food tables. Preflight validation, invalid-row filtering, and transaction boundaries reduce the chance of leaving the database in a half-loaded state.

The imported data directly supports backend runtime behavior. Ingredients provide the searchable catalog, ingredient nutrition records provide per-100g macro values, ingredient unit records provide household conversion data, recipe ingredients connect recipes to quantities, and category/diet fields support filtering and menu recommendation. Backend services such as recipe nutrition calculation, unit conversion, recommendation scoring, inventory forms, consumption preview, and shopping-list workflows all depend on this prepared dataset.

14.3 Data Quality Risks and Design Rationale

Food datasets are naturally imperfect. Ingredient names may vary, nutrition rows may be incomplete, units may not be comparable across ingredients, and spreadsheet formatting may introduce numeric errors. The project mitigates these risks through conservative fuzzy matching, relationship validation, dry-run support, default operational values, runtime fallbacks, and transaction-safe import.

The pipeline does not claim to make the dataset flawless. Instead, it reduces the probability that data problems will break user-facing workflows or produce misleading recommendations. This follows the same principle as the recommendation architecture: unreliable input should be cleaned, validated, and bounded before it reaches the user.

15. Frontend Architecture and User Interface

The frontend is a React, TypeScript, and Vite single-page application. Its role is to provide the user-facing workflows for authentication, profile management, inventory management, recipe

browsing and editing, AI-assisted recommendation, consumption logging, notifications, and administrative review. The frontend does not duplicate core backend business logic; instead, it consumes structured REST APIs and focuses on presentation, routing, form state, feedback, and workflow coordination.

The application follows a feature-oriented structure. User-facing features are grouped under frontend/src/features, shared UI elements under frontend/src/shared, cross-cutting infrastructure under frontend/src/infrastructure, and API wrappers under frontend/src/services. This organization keeps workflow-specific code close to the screens that use it while allowing authentication, HTTP configuration, theme state, toast messages, and service access to be reused across the application.

15.1 Technology and Structure Summary

Area	Implementation role
React and TypeScript	Component-based UI development with static typing for DTOs, props, and shared domain models.
Vite	Development server and production build tooling. The build process includes TypeScript compilation before bundling.
React Router	Public, authenticated, and admin route organization.
Axios	Centralized HTTP communication through a shared API client.
Keycloak JS	Browser-side authentication integration with the external identity provider.
i18next	Localization support for navigation, forms, prompts, and user feedback.
Tailwind CSS and shared UI components	Responsive layout, reusable interaction patterns, loading states, empty states, and modal workflows.
Service registry and contexts	Shared access to authentication, HTTP, theme, toast, definitions, and global UI workflows.

The main feature areas include dashboard, recommendations, inventory, consumption, recipes, profile, notifications, admin, about, and landing-page modules. Examples of feature responsibilities are summarized in Table 15.2.

Feature area	Main responsibilities
Dashboard	Overview, daily summary, inventory shortcuts, and shopping-list access.
Recommendations	AI model selection, inventory context, craving input, recommendation results, menu recommendation, rating, and cooked markers.
Inventory	Group management, item management, invitations, member handling, conversion preview, stock consumption, and shopping list.

Feature area	Main responsibilities
Recipes	Recipe listing, recipe details, draft/edit workflows, image upload, favorites, ratings, and approval-related actions.
Consumption	Smart consumption logging, nutrition preview, meal type selection, source selection, and history.
Profile	User preferences, body metrics, diet configuration, allergies, disliked ingredients, and settings.
Notifications and admin	Notification actions, pending workflows, privileged user, ingredient, and recipe management screens.

15.2 Application Shell and Authentication Flow

The root application composes the main providers before rendering feature screens. The service registry registers shared services such as authentication, logging, and the HTTP client. The authentication provider initializes the configured authentication service, tracks whether authentication is ready, stores the current user, and exposes login, registration, and logout functions.

Protected routes are handled by a private-route component. If a user is not authenticated, the frontend shows an authentication prompt rather than rendering a protected workflow. If a route requires an administrative role, the route also checks the current user's roles and displays an access-denied state when appropriate. These frontend checks improve usability, while the backend remains the final authority for authorization.

API calls are centralized through typed service modules. A shared Axios instance attaches the current bearer token to outgoing requests when authentication is available. Feature components therefore do not need to manually manage token injection. This keeps low-level HTTP concerns separate from screen logic and reduces duplicated request handling across the codebase.

15.3 Workflow-Oriented UI Design

The frontend is designed as a functional application rather than a static landing page. The main layout provides the navigation frame, sidebar behavior, profile menu, notification dropdown, language switching, theme switching, and access to shared modal workflows. Global UI context is used for cross-feature actions such as opening settings, displaying recipe details, or starting consumption logging from a recommendation result.

The recommendation page is the most representative workflow because it combines several bounded contexts. It loads user profile data, inventory groups, recommendation history, AI key state, and rating information. It then allows the user to select a recommendation mode, choose inventory context, enter cravings, request recipe or menu recommendations, inspect explanations, rate results, and mark recipes as cooked.

The inventory interface follows a similar workflow-based design. A dedicated inventory hook coordinates selected group state, item loading, invitation state, unit conversion preview, shopping-list loading, member search, and modal visibility. This prevents a single page component from becoming responsible for every asynchronous state transition.

15.4 Type Safety, Localization, and Feedback

Shared TypeScript types define important frontend data shapes such as users, recipes, inventory groups, recommendation responses, validation errors, enum definitions, and consumption models. This reduces accidental mismatch between frontend requests and backend DTOs and makes feature refactoring safer.

Localization is handled with react-i18next, and language switching is exposed through the application layout. Theme state is also centralized, allowing the application to support consistent light and dark modes. User feedback is provided through toast messages, loading states, empty states, validation messages, and access-denied states. These patterns are important because the platform contains many asynchronous workflows where users need clear confirmation or recovery paths.

15.5 Frontend Design Rationale

The frontend architecture supports the same separation-of-concerns principle used in the backend. Infrastructure concerns such as authentication, HTTP access, theme state, and toast feedback are initialized once and reused. Feature modules then build domain-specific workflows on top of that foundation. This makes the user interface maintainable while still supporting complex interactions across inventory, recipes, recommendations, consumption history, notifications, and administration.

Overall, the frontend demonstrates a connected full-stack client: it protects routes, consumes typed backend APIs, supports multilingual interaction, coordinates cross-feature modals, and presents recommendation and inventory workflows in a way that ordinary users can operate repeatedly.

16. REST API Design

The backend exposes the platform through a versioned REST API under /api/v1. The API is implemented in the application module with Spring MVC controllers, DTOs, mappers, validation annotations, JWT principal injection, and a global exception handler. The design is mostly resource-oriented, but it also uses explicit action endpoints for workflows such as recommendation generation, recipe approval, invitation acceptance, and marking a recommendation as cooked.

The React frontend consumes these APIs through a shared Axios client configured with the /api base path. Bearer tokens are attached by the frontend interceptor and validated by the Spring Security resource server.

16.1 API Organization

API concern	Design summary
Controller grouping	Controllers are organized by domain area, including users, recipes, recommendations, ingredients, inventory groups, invitations, consumptions, notifications, definitions, and admin operations.
Versioning	Most production endpoints use /api/v1/..., allowing future API changes without immediately breaking existing clients.
Resource routes	Resource-oriented paths are used where possible, such as /api/v1/users, /api/v1/recipes, /api/v1/ingredients, and /api/v1/notifications.
Nested inventory routes	Inventory item operations are scoped under inventory groups because items belong to a selected group and require group-level authorization.
Action routes	Commands such as generating recommendations, approving recipes, accepting invitations, rating recommendations, and marking recipes as cooked use explicit action endpoints.
File uploads	Multipart endpoints support profile and recipe image upload while keeping storage details behind backend services.
Definitions	Enum-definition endpoints provide backend-supported values and localized labels for frontend dropdowns and selectors.

This organization keeps API responsibilities close to the workflows they expose. For example, inventory group CRUD, member operations, invitation handling, stock consumption, and shopping-list retrieval are handled through inventory-related controllers rather than a generic catch-all endpoint.

16.2 Request, Response, and Security Contracts

The API uses DTO classes rather than exposing JPA entities directly. Request DTOs define accepted client input, while response DTOs shape data for frontend workflows. This is important for recommendation responses, inventory responses, user profile data, and recipe views, where nested domain objects must be controlled before leaving the backend.

Protected endpoints receive the authenticated JWT through @AuthenticationPrincipal Jwt. The backend uses the JWT subject as the authenticated user id and validates that users cannot act on another user's profile or private resources. Administrative endpoints are additionally protected by role checks. Frontend route guards improve the user experience, but backend authorization remains the final enforcement layer.

The API also centralizes error responses through a global exception handler. Domain exceptions, validation failures, missing resources, data-integrity errors, unreadable request bodies, and

unexpected exceptions are converted into consistent JSON responses. The frontend service layer maps these responses into application-level errors and validation messages.

16.3 Frontend API Consumption

Frontend API access is centralized in typed service modules. Recommendation generation, menu recommendation, recommendation history, rating, cooked markers, recipe operations, inventory actions, consumption logging, and definition loading are called through service functions instead of being embedded directly in UI markup.

This pattern keeps the route contract maintainable. If a backend path or response shape changes, the service layer can be updated in one place while feature components continue to focus on form state, loading state, feedback, and rendering.

16.4 REST API Design Rationale

The REST API acts as the contract between the React frontend and the Spring Boot backend. Versioned paths provide stability, DTOs protect internal domain models, JWT principal extraction binds operations to authenticated users, role checks protect administrative workflows, nested routes express ownership, action routes model command-like workflows, and global exception handling gives the frontend a predictable error format.

Overall, the API provides access to the platform's core capabilities while preserving validation, authorization, and domain rules on the server side.

17. Testing and Quality Assurance

Quality assurance in the project is handled through automated backend tests, integration-oriented test infrastructure, frontend build checks, and defensive runtime behavior. Because the platform combines authentication, inventory, recommendation, AI provider integration, nutrition calculation, recipe workflows, and data import logic, testing is distributed across multiple layers rather than concentrated in a single test category.

At the time this section was drafted, the backend source tree contained 46 Java test classes under src/test/java. The important point for the report is not the exact number, but the fact that tests cover multiple architectural layers and high-risk workflows.

17.1 Testing Strategy and Coverage Areas

The backend testing strategy follows the modular structure of the application. Pure domain rules are tested close to the domain layer, HTTP behavior is tested through MockMvc, persistence behavior is tested through repository and integration tests, and shared infrastructure is tested through reusable base classes. Table 17.1 summarizes the main quality-assurance areas.

Area	Purpose	Examples of verified behavior
Domain and service tests	Verify business rules independently from the UI.	recommendation scoring, unit conversion, consumption logging, inventory deduction, recipe lifecycle transitions.
Repository and persistence tests	Verify entity relationships and query behavior against realistic persistence behavior.	users and inventory groups, recipes and ingredients, daily consumption, recommendation history, recipe versions.
Controller tests	Verify API routing, validation, serialization, authentication context, and HTTP status behavior.	user profile upsert, identity relinking, recommendation requests, protected routes.
DTO, mapper, and exception tests	Protect the API contract between backend and frontend.	response shapes, validation fields, global error response objects.
Infrastructure tests	Support integration scenarios involving external-like dependencies.	PostgreSQL, Keycloak, MinIO initialization, dynamic test properties.
Frontend checks	Catch client-side type and lint issues before build.	TypeScript compilation, ESLint rules, import and hook correctness.
Runtime safeguards	Reduce risk when invalid input or external failures occur.	structured API errors, fallback recommendations, insufficient-stock exceptions, recipe approval states, import preflight checks.

This layered approach is appropriate because different project risks require different verification methods. A pure unit test is suitable for deterministic scoring or unit conversion, while authentication and persistence behavior need Spring and container-backed integration support.

17.2 Testcontainers-Based Integration Infrastructure

The project uses Testcontainers to make integration tests closer to the actual runtime environment. Instead of relying only on shallow in-memory substitutes, the shared test infrastructure can register PostgreSQL, Keycloak, and MinIO-related properties for Spring tests.

backend/modules/01-infrastructure/infrastructure-test/src/main/java/com/mealapp/infrastructure/test/AbstractSpringTest.java:

```
@SpringBootTest(
    classes = TestApplication.class,
    webEnvironment = SpringBootTest.WebEnvironment.MOCK
)
@Import({TestDataSourceConfig.class, MinioInitConfig.class})
@ActiveProfiles({"local", "junit"})
@Testcontainers
public abstract class AbstractSpringTest extends Assertions {
```

```
    static MinIOContainer minio = new MinIOContainer(MINIO_IMAGE)
        .withUserName("admin")
        .withPassword("MealApp123!");

    @DynamicPropertySource
    static void registerContainerProperties(DynamicPropertyRegistry registry) {
        registry.add("com.mealapp.infrastructure.storage.minio.endpoint", minio::getS3URL);
        registry.add("spring.datasource.url", () -> PostgresSingleton.INSTANCE.getJdbcUrl());
        registry.add("spring.datasource.username", () -> PostgresSingleton.INSTANCE.getUsername());
        registry.add("spring.datasource.password", () -> PostgresSingleton.INSTANCE.getPassword());
    }
}
```

This base class reduces repeated setup across integration tests and keeps tests aligned with the containerized architecture. PostgreSQL verifies relational behavior, Keycloak supports authentication-related scenarios, and MinIO-like configuration supports storage-dependent workflows.

17.3 Critical Workflow Verification

The most important tested workflows are the ones that connect multiple domains. User identity tests verify profile upsert, authenticated access, and identity relinking by email. Recommendation tests verify fallback behavior, disliked ingredient handling, inventory-aware ranking, AI response handling, and recent-history diversity. Consumption tests verify that logging a recipe can calculate nutrition and deduct inventory when the selected source is an inventory group. Unit conversion tests cover aliases, Turkish character normalization, liquid and solid conversions, and ingredient-specific unit weights.

These tests matter because the core value of the platform depends on cross-domain correctness. A recommendation result is only useful if authentication, profile data, inventory state, recipe ingredients, nutrition values, and fallback behavior work together consistently.

17.4 Frontend Quality Checks

The frontend quality gates are defined through package scripts. The build process runs TypeScript compilation before Vite bundling, and the lint process checks TypeScript and React code with strict warning handling. These checks help detect type mismatches, invalid imports, React hook issues, and general lint violations before deployment.

The current frontend quality strategy is useful but not complete. The frontend still relies more heavily on TypeScript, linting, and manual workflow verification than on automated component or browser-level tests.

17.5 Limitations and Future QA Work

The current QA approach provides a strong backend foundation, but several areas should remain future priorities: broader frontend component tests, end-to-end browser tests for login and recommendation workflows, visual regression checks for responsive UI states, API contract tests between backend DTOs and frontend types, performance tests for larger recommendation candidate pools, and import-pipeline tests using controlled sample workbooks.

Overall, the testing strategy reflects the architecture of the project. Domain rules are tested close to their services, HTTP behavior is tested through controller tests, integration dependencies are represented with Testcontainers, frontend correctness is supported by static checks, and runtime safeguards handle invalid states or external service failures.

18. Deployment and Runtime Environment

The project is designed to run as a full-stack web application with containerized infrastructure. The runtime environment contains a Spring Boot backend, a React frontend served through Nginx, PostgreSQL for relational persistence, Keycloak for OAuth2/OIDC identity management, and MinIO for S3-compatible object storage. Docker Compose coordinates these services for local development, testing, and project demonstration.

The deployment should not be described as a production microservice platform. It is more accurately a modular full-stack system where the backend is a modular Spring Boot application and the surrounding runtime dependencies are started as separate Docker services.

18.1 Runtime Topology

The Compose configuration defines infrastructure-only and full-stack profiles. Developers can start only PostgreSQL, Keycloak, and MinIO while running backend or frontend locally, or they can start the complete stack including backend and frontend containers. All services communicate over a dedicated Docker bridge network.

Service	Runtime role	Local exposure or integration
PostgreSQL	Stores application data and also hosts the Keycloak database in the local setup.	Exposed on 5432; initialized with init-db.sql.
Keycloak	Provides OAuth2/OIDC login, JWT issuing, realm configuration, roles, and identity data.	Exposed on 8080; imports the project realm and mounts the custom theme.
MinIO	Provides S3-compatible object storage for files such as recipe and profile images.	API exposed on 9000; console exposed on 9001.
Backend	Runs the Java 21 Spring Boot application, REST API, Flyway migrations, security validation, and business workflows.	Exposed on 8081; connects to PostgreSQL, Keycloak, MinIO, and optional AI providers.
Frontend	Serves the compiled React application through Nginx.	Exposed on 3030; proxies /api requests to the backend service.

Named volumes are used for PostgreSQL and MinIO so that database and object-storage data can persist across container restarts during local development.

18.2 Backend and Frontend Containerization

The backend Dockerfile uses a multi-stage build. The build stage compiles the application with Gradle using Java 21, and the runtime stage runs the generated Spring Boot JAR with java -jar. The local .sdkmanrc may pin a specific SDK distribution for developer machines, but the report describes the project requirement generally as Java 21 to avoid unnecessary vendor-specific claims.

The frontend Dockerfile also uses a multi-stage build. The first stage installs dependencies and builds the Vite application, while the final stage uses Nginx to serve the compiled static assets. The Nginx configuration supports React Router by falling back to index.html for client-side routes and proxies API calls to the backend container.

During local frontend development, Vite proxies /api requests to the backend running on the developer machine. This mirrors the same path convention used by the production-style Nginx container, allowing frontend code to call /api/... consistently.

18.3 Runtime Configuration

The backend reads configuration from Spring properties and environment variables. Important runtime settings include server port, datasource connection, JWT issuer URI, JWK set URI, Flyway settings, AI provider configuration, encryption keys, super-admin integration, and MinIO storage settings.

The distinction between browser-visible and container-internal URLs is important. For example, the browser may access Keycloak through localhost, while the backend container resolves Keycloak through the Compose service name. Environment variables allow the same backend artifact to run in local and containerized modes without source-code changes.

Flyway is enabled at runtime and reads migrations from classpath:db/migration. This makes schema evolution part of application startup and keeps database changes versioned, repeatable, and reviewable.

18.4 Deployment Limitations and Production Hardening

The current deployment setup is suitable for local development, demonstration, and controlled testing. It is not a complete production cloud architecture. A production-oriented version would require stronger operational controls, including managed secrets, TLS termination, production Keycloak settings, hardened MinIO credentials and bucket policies, health checks, restart policies, database backups, centralized logging, metrics, and CI/CD image management.

These limitations do not reduce the value of the Compose setup for the graduation project. The current runtime demonstrates how the application components work together in a reproducible environment: modular Java backend, compiled React frontend, relational persistence, identity management, object storage, schema migration, and configurable AI provider integration.

19. Risks, Limitations, and Mitigations

Every software project has technical and operational risks. This platform combines identity management, nutrition estimates, shared inventory, external AI providers, file storage, imported datasets, and containerized runtime services. Therefore, the project should be evaluated as a graduation-level full-stack system, not as a medical nutrition product, fully autonomous AI agent, or production-hardened cloud platform.

The main risk strategy is layered mitigation: deterministic backend logic handles safety-critical decisions, AI remains optional, imported data is cleaned before use, authorization is enforced on the backend, and deployment limitations are documented honestly. Table 19.1 summarizes the main risks, current mitigations, and remaining limitations.

Risk area	Description	Implemented mitigation	Remaining limitation or future work
AI provider reliability	External AI services may fail, respond slowly, return invalid JSON, or become unavailable because of network or API-key problems.	The backend first filters and ranks candidates deterministically. FREE mode and fallback recommendations keep the feature usable without AI.	Future work can add timeout tuning, provider health checks, retry policies, and clearer user-facing provider status.
AI hallucination and trust	A language model may generate convincing but incorrect explanations, ingredients, or nutrition statements.	AI receives only a curated candidate pool. Recipe safety, allergy filtering, diet compatibility, and ranking are handled by backend logic.	Explanation templates and post-processing could be made stricter so AI wording is composed only from verified backend facts.
Nutrition accuracy	Calories and macronutrients are estimates based on ingredient data, servings, quantities, and conversions.	The system uses ingredient-level nutrition records, gram-based aggregation, serving safeguards, and neutral scoring when nutrition data is incomplete.	The system should not be presented as medical-grade dietary advice. Clinical use would require expert-validated datasets and medical constraints.
Dataset quality	Food datasets may contain duplicate ingredients, missing values, invalid relationships, inconsistent units, or spreadsheet formatting issues.	Python utilities normalize names, handle safe duplicate merging, validate relationships, support dry-run import, and write data transactionally.	Automated cleaning cannot fully understand food semantics; manual review remains useful for high-quality production data.

Risk area	Description	Implemented mitigation	Remaining limitation or future work
Unit conversion	Household units vary by ingredient, language, preparation method, and density.	The unit converter uses ingredient-specific weights, aliases, density support, Turkish character normalization, and fallback values.	Some conversions remain approximate; future admin tools could refine conversion tables from observed data.
Security and authorization	Unauthorized users could attempt to access private profiles, inventory groups, recipe revisions, or admin functions.	Keycloak, Spring Security JWT validation, admin role checks, ownership checks, group membership checks, and frontend route guards are used.	Production deployment would need TLS, stronger CORS policy, audit logging, hardened Keycloak settings, and infrastructure access controls.
AI API key privacy	User-provided provider keys could be exposed if stored or logged incorrectly.	API keys are encrypted before local storage and decrypted only when needed for provider calls. They are separate from application authentication.	Production systems should prefer server-side secret storage or a managed secret vault instead of long-term client-side storage.
Inventory consistency	Shared inventory updates, consumption logging, and shopping-list generation may produce incorrect stock if quantities are mishandled.	The system enforces group membership, unique ingredient rows per group, unit conversion, insufficient-stock exceptions, and controlled deduction logic.	Heavy concurrent household usage could require optimistic locking, stronger transaction isolation, or version fields.
Recipe moderation	User-submitted recipes may contain incomplete, incorrect, or inappropriate content.	Draft, pending, approved, rejected, and superseded statuses keep public content moderated. Admin approval is required for publication.	Moderation quality still depends on human review; automated validation can support but not replace content judgment.
External service availability	PostgreSQL, Keycloak, MinIO, and optional AI providers are required for different parts of the system.	Docker Compose, Testcontainers, storage abstraction, Flyway migrations, and AI-free recommendation mode reduce local runtime risk.	Production use would require monitoring, backups, health checks, restart policies, and redundancy.

Risk area	Description	Implemented mitigation	Remaining limitation or future work
Performance	Recommendation generation may become expensive with larger recipe, inventory, rating, and history data.	Candidate pools are bounded, unsafe recipes are filtered early, AI receives curated data, and database evolution supports indexing.	Future work can add query profiling, caching, pagination tuning, asynchronous jobs, and load testing.
Frontend reliability	The frontend coordinates authentication, inventory, recommendations, modals, notifications, and API-key state.	Typed services, feature hooks, shared contexts, loading states, toast feedback, route guards, TypeScript, and lint checks reduce UI risk.	Broader component and end-to-end tests are still needed for critical workflows.
Deployment and secrets	The Compose setup uses local development assumptions and placeholder-style configuration.	Environment-driven configuration, Dockerized services, multi-stage builds, and documented boundaries support reproducible demonstration.	A production version would require managed secrets, TLS, production Keycloak settings, backups, observability, and CI/CD image management.
Testing coverage	Existing tests cannot cover every UI path, imported dataset case, or high-load scenario.	Backend tests cover multiple layers, and runtime safeguards handle invalid states and provider failures.	Future QA should add browser E2E tests, visual regression checks, API contract tests, and import tests with controlled workbooks.
Ethical and scope boundaries	Nutrition-related recommendations may be misunderstood as professional medical advice.	The report frames the system as a meal-planning and software-engineering project, with AI as optional assistance rather than medical authority.	User-facing disclaimers and stronger policy language would be needed before broader public deployment.

Overall, the project manages risk by keeping critical decisions inside deterministic backend services, treating AI as an enhancement layer, validating data before use, enforcing authenticated access, and clearly separating demonstration deployment from production hardening. This posture is appropriate for the current project stage and leaves a concrete path for future improvement.

20. Conclusion and Future Work

This project delivered an AI-powered personalized meal recommendation platform as a full-stack CENG 408 graduation project. The system addresses a practical meal-planning problem by connecting recommendations to real ingredients, dietary restrictions, nutritional context, shared inventory, recent consumption, and user feedback. The final implementation combines a modular Spring Boot backend, React and TypeScript frontend, PostgreSQL persistence, Keycloak authentication, MinIO object storage, Flyway schema migration, and Python-based dataset preparation utilities.

The most important engineering decision was to keep AI controlled. The platform does not allow an external AI provider to become the sole authority for meal recommendation. Instead, backend services retrieve safe recipe candidates, apply allergy and diet filtering, calculate nutritional context, score recipes with deterministic signals, and use AI only as an optional explanation or menu-selection layer. When AI is unavailable, invalid, or not selected, deterministic fallback recommendations remain available.

20.1 Project Outcomes

The implemented system satisfies the main objectives of the project. It provides authenticated user profile management, inventory groups, shared household inventory, recipe browsing and editing, recipe approval workflows, recipe and menu recommendation, nutrition calculation, consumption tracking, notifications, administrative operations, dataset preparation utilities, and Docker-based runtime support.

The main technical contributions are summarized as follows:

- a hybrid recommendation workflow with deterministic ranking and optional AI assistance;
- inventory-aware recommendation based on available ingredients;
- shared inventory groups with invitations and member management;
- recipe lifecycle management with draft, pending, approved, rejected, and superseded states;
- nutrition calculation based on ingredient-level macro values and gram conversion;
- daily consumption logging with nutrition summaries and optional inventory deduction;
- a maintainable full-stack architecture with modular backend boundaries and feature-oriented frontend structure.

These outcomes show that the project is more than a recipe search interface. It is an integrated platform where recommendation, inventory, nutrition, identity, moderation, data preparation, and user feedback operate together.

20.2 Limitations and Future Work

The current system has clear limitations. Nutrition values and calorie targets are estimates and should not be treated as medical advice. Unit conversion is practical but approximate, especially for household units that vary by ingredient preparation. The deployment environment is suitable

for local demonstration but not production hardening. Backend testing is stronger than frontend automated testing, and broader end-to-end coverage would improve confidence.

Future work can improve the platform in several directions:

Direction	Possible improvement
Recommendation personalization	Add stronger long-term preference learning, collaborative filtering, seasonal suggestions, budget-aware scoring, and household-level recommendation history.
Nutrition support	Add macro targets, micronutrient tracking, sodium or sugar limits, and more precise serving-size modeling with appropriate disclaimers.
Data quality	Expand validation reports, duplicate-review tools, ingredient taxonomy management, and controlled import tests.
Frontend testing	Add component and end-to-end tests for login, inventory updates, recommendation generation, recipe submission, approval, and consumption logging.
Production deployment	Add managed secrets, TLS, health checks, backups, observability, CI/CD image management, and stronger Keycloak/MinIO hardening.
Analytics and user experience	Add dashboards for consumption trends, pantry usage, recommendation success, nutrition patterns, and shopping-list behavior.

20.3 Final Evaluation

Overall, the project demonstrates a complete software engineering solution for personalized meal recommendation. It combines backend domain modeling, secure identity integration, data processing, frontend workflow design, AI provider abstraction, deterministic fallback behavior, testing infrastructure, and containerized runtime support.

The final system is valuable because it connects recommendation to practical everyday constraints. It can suggest meals based on what users have, what they cannot eat, what they prefer, what they recently consumed, and what nutritional goals they follow. By keeping AI optional and bounded by verified application data, the platform remains useful even without external provider access and avoids presenting generative AI output as unchecked truth.

The project therefore achieves its main goal: building a maintainable, extensible, and realistic AI-powered meal recommendation platform that demonstrates both modern full-stack development and responsible AI-assisted system design.

21. References

[1] F. Ricci, L. Rokach, and B. Shapira, editors, *Recommender Systems Handbook*, 3rd ed. Springer, 2022. DOI: 10.1007/978-1-0716-2197-4.

[2] R. Burke, "Hybrid Recommender Systems: Survey and Experiments," *User Modeling and User-Adapted Interaction*, vol. 12, pp. 331-370, 2002. DOI: 10.1023/A:1021240730564.

[3] S. Abhari, R. Safdari, L. Azadbakht, K. B. Lankarani, Sh. R. Niakan Kalhori, B. Honarvar, Kh. Abhari, S. M. Ayyoubzadeh, Z. Karbasi, S. Zakerabasali, and Y. Jalilpiran, "A Systematic Review of Nutrition Recommendation Systems: With Focus on Technical Aspects," *Journal of Biomedical Physics and Engineering*, vol. 9, no. 6, pp. 591-602, 2019. DOI: 10.31661/jbpe.v0i0.1248.

[4] K. Kim, D. Park, M. Spranger, K. Maruyama, and J. Kang, "RecipeBowl: A Cooking Recommender for Ingredients and Recipes Using Set Transformer," *IEEE Access*, 2021. DOI: 10.1109/ACCESS.2021.3120265.

[5] Y. S. Soekamto, A. Lim, L. C. Limanjaya, Y. K. Purwanto, S.-H. Lee, and D.-K. Kang, "Pic2Plate: A Vision-Language and Retrieval-Augmented Framework for Personalized Recipe Recommendations," *Sensors*, vol. 25, no. 2, article 449, 2025. DOI: 10.3390/s25020449.

[6] U.S. Department of Agriculture, Agricultural Research Service, "FoodData Central." Available: https://fdc.nal.usda.gov/.

[7] U.S. Department of Agriculture and U.S. Department of Health and Human Services, *Dietary Guidelines for Americans, 2020-2025*. Available: https://odphp.health.gov/our-work/nutrition-physical-activity/dietary-guidelines/previous-dietary-guidelines/2020.

[8] D. Tsolakidis, L. P. Gymnopoulos, and K. Dimitropoulos, "Artificial Intelligence and Machine Learning Technologies for Personalized Nutrition: A Review," *Informatics*, vol. 11, no. 3, article 62, 2024. DOI: 10.3390/informatics11030062.

[9] Y. Zhang and X. Chen, "Explainable Recommendation: A Survey and New Perspectives," *Foundations and Trends in Information Retrieval*, 2020. Available: https://arxiv.org/abs/1804.11192.

[10] Spring, "Spring Boot Reference Documentation." Available: https://docs.spring.io/spring-boot/reference/.

[11] Meta Open Source, "React Documentation." Available: https://react.dev/.

[12] PostgreSQL Global Development Group, "PostgreSQL Documentation." Available: https://www.postgresql.org/docs/.

[13] United Nations Environment Programme, *Food Waste Index Report 2024*. Available: https://www.unep.org/resources/publication/food-waste-index-report-2024.

[14] U.S. Environmental Protection Agency, "Preventing Wasted Food at Home." Available: https://www.epa.gov/recycle/reducing-wasted-food-home.

[15] Spring Security, “OAuth2 Resource Server JWT Documentation.” Available: https://docs.spring.io/spring-security/reference/servlet/oauth2/resource-server/jwt.html.

[16] Keycloak, “Server Administration Guide.” Available: https://www.keycloak.org/docs/latest/server_admin/.

[17] Docker, “Docker Compose Documentation.” Available: https://docs.docker.com/compose/.

[18] Redgate, “Flyway Documentation.” Available: https://documentation.red-gate.com/flyway.

[19] MinIO, “MinIO Object Storage Documentation.” Available: https://min.io/docs/minio/linux/index.html.

[20] Testcontainers, “Testcontainers for Java Documentation.” Available: https://java.testcontainers.org/.

[21] Vite, “Vite Documentation.” Available: https://vite.dev/.

[22] NGINX, “NGINX Documentation.” Available: https://nginx.org/en/docs/.